COMP 488 · CLOUD COMPUTING, DEVOPS, AND AI · PROJECT PRESENTATION

InferScale

One ML inference service on three clouds, managed with cmx vm

Local · Multipass | OpenStack · Chameleon KVM@TACC | Public · AWS EC2

Presenter
Khaja Shabbir Ahmed

Instructor
Gregor von Laszewski

Date
October 2026

THE PROBLEM AND THE IDEA

Every cloud works differently

THE PROBLEM

The same AI service often has to run on more than one cloud, but every cloud creates VMs, hands out IPs and cleans up in its own way.

Deployments are hard to repeat. Every cloud needs its own steps.

Clouds are hard to compare. Different steps blur a side-by-side comparison.

THE IDEA: INFERSCALE

Package one three-tier AI service once, deploy it the same way on three clouds with cmx vm, and measure the clouds side by side.



CLOUD COMPUTING, DEVOPS AND AI

One feature for each course area

Cloud Computing

Three-tier VMs on a local, an OpenStack and a public cloud: IPs, security groups, leases, cost.

FEATURE

Multi-cloud failover

client.py tries AWS, then Chameleon, then Multipass. Target: 99 % succeed with AWS deleted.

DevOps

GitHub flow, CI with tests and security scans, one deploy script, smoke tests, clean-up.

FEATURE

Live monitoring + alerts

Prometheus and Grafana watch all three clouds. Target: every 5 s; an alert within 2 min.

AI

A DistilBERT sentiment model served on CPU, its response time measured on every cloud.

FEATURE

AI review filter

/filter keeps only confident reviews with a chosen label. Target: 18 of 20 correct.

OBJECTIVES AND REQUIREMENTS

What the project has to achieve

Eight objectives

1. Package the three tiers as containers
2. Run on three clouds, all VMs through cmx vm
3. Deploy with one command: deploy.sh
4. Measure and compare the three clouds
5. Improve cmx vm with small, reviewed PRs
6. AI review filter
7. Live monitoring with alerts
8. Multi-cloud failover

Quality targets

< 1 s server p95 for one review	**≥ 99 %** of requests succeed	**< 15 min** to deploy on a cloud
$0 out of pocket	**443** the only open port, with login	**2 GB** VM on each cloud

IN SCOPE

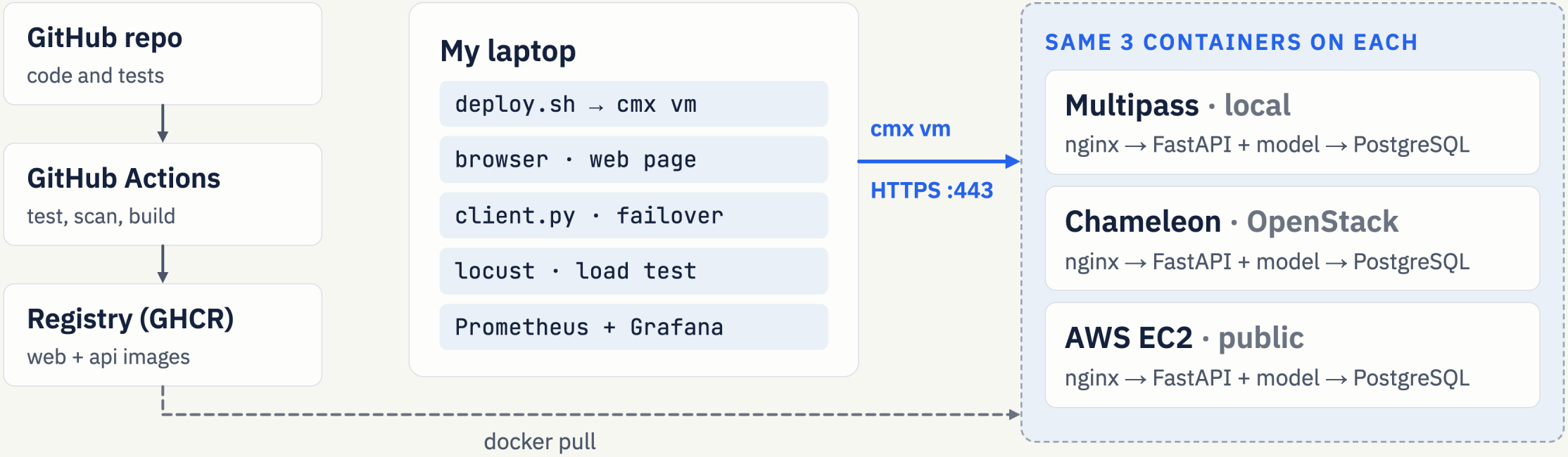
Three tiers with a login over HTTPS, three clouds, a deploy script built on cmx vm, CI, load tests, small cmx vm fixes, three features

OUT OF SCOPE

Training models, Kubernetes, GPUs, high availability, other public clouds, a load balancer, sign-up pages, syncing data between clouds

ARCHITECTURE

Three tiers, three clouds, one laptop



Only nginx is public. The API and the database talk on a private Docker network.

THREE CLOUDS AND CMX VM

One workflow, three clouds

	Multipass	Chameleon KVM@TACC	AWS EC2
Kind	Local	OpenStack (research)	Public
VM	1 vCPU, 2 GB	m1.small: 1 vCPU, 2 GB	t3.small: 2 vCPU, 2 GB
Reached by	Local IP	Floating IP	Public IP
Extra step	None	Lease (2 h), ended by destroy.sh	Delete after every session
Status	Lifecycle tested	Tested on Oct 14	Account not set up yet

DEPLOY.SH: THE SAME STEPS ON EVERY CLOUD

Set the cloud → **Start the VM** → **Install Docker** → **Create secrets** → **Start 3 tiers** → **Smoke test** → **Load test** → **destroy.sh**

Only the cloud name changes. Secrets are made on the VM or kept on my laptop, never in Git.

DEVOPS

From a push to a monitored service

Source
GitHub, PRs

→

Test + scan
pytest, gitleaks, pip-audit

→

Build
web + api images

→

Deploy
deploy.sh + cmx vm

→

Verify
/api/health, Locust

→

Monitor
Prometheus, Grafana

→

Clean up
destroy.sh

On every push

- Unit and container tests, a secret scan and a dependency check
- Two images, web and api, tagged with the Git commit
- CPU-only PyTorch with the model built in, so the image stays small

Live monitoring

- Prometheus on my laptop reads each cloud every 5 seconds
- Grafana: requests, errors and p95 latency per cloud
- Alerts: CloudDown after 1 minute, HighErrorRate above 5 %

SECURITY AND RELIABILITY

Protected and robust by design

Security

Login for every user. nginx checks four fixed accounts: owner, demo, client and prometheus.

HTTPS only. A self-signed certificate per VM, pinned by my laptop; the VM keeps only hashes.

One public port. Only 443; api and db stay on a private Docker network; 50 requests/s limit.

Protected database. Password made on the VM, parameterized SQL, each user sees only their history.

No secrets in Git. Passwords stay in ~/.config/; CI scans for secrets and vulnerable packages.

Reliability

Clear errors. Bad input gets 422, a wrong login 401, too many requests 429.

Self-healing containers. Each container restarts by itself after a crash.

Safe deployments. deploy.sh stops at the first error and retries the health check.

Database outages. The API answers 503 instead of crashing, and the db container restarts.

Failover and alerts. client.py moves to the next cloud; CloudDown and HighErrorRate fire.

EVALUATION

How success is measured

Metric	How it is measured	Target
Deploy time	Timer in deploy.sh: cmx vm start → first /predict	Under 15 minutes per cloud
Latency p50 / p95	Locust end to end; Prometheus on the server	Reported per cloud
Success rate	Locust, 60 s, 10 users	At least 99 %
Cost per session	Provider pricing and allocation used	Under $1 per AWS session
Same script everywhere	Run deploy.sh with each cloud	No changes to the steps
Review filter accuracy	/filter on 20 reviews I label by hand	At least 18 of 20 correct
Live monitoring	Grafana during a load test	All clouds, every 5 seconds
Alerts	Delete one cloud’s VM	CloudDown within 2 minutes
Failover	200 requests, AWS VM deleted halfway	At least 99 % succeed
Security	No or wrong password; client on /api/metrics	All 401; /api/health answers
History	5 analyses, then restart all containers	All 5 still listed

STATUS, BUDGET AND RISKS

Where the project stands

Done · Sep 23 – Oct 14

Topic and design; cmx vm on Multipass and Chameleon, including the new cmx vm ip command

Next · Oct 15 – Oct 29

AWS account and first VM (Oct 22); the service with its database, login and tests (Oct 29)

Planned · Oct 29 – Dec 10

CI (Nov 5), deploy.sh (Nov 12), monitoring and failover (Nov 19), measurements (Dec 3), final report and presentation (Dec 10)

$0

out of pocket

Free tiers, class allocation and AWS credits

Main risks

- Leases left running → end every session
- AWS left running → delete, billing alert
- Over 2 GB → small PostgreSQL, swap
- History lost → destroy.sh exports it

ISSUES

Open questions and known gaps

AWS is not ready yet

I still need the account and credentials, and I have not tested whether the cmx vm aws provider supports a public IP and a security group rule.

Chameleon clean-up

Deleting a VM does not end its lease or release its floating IP, so every session needs a manual clean-up step to avoid wasting allocation.

Is 2 GB enough?

DistilBERT, PostgreSQL and Docker together may need swap or a smaller model.

Which Chameleon code?

cmx vm has two versions for Chameleon: the dev2 branch and the feature/chameleon-native-chi branch. I need to know which one the project should build on.

Failover setup

Failover runs in the client, not a load balancer. The cloud IPs change with every deployment, so deploy.sh must update the client's config file and Prometheus's target list each time.

A fair comparison

The VMs are not identical (t3.small has 2 vCPUs, m1.small has 1), so the results will note the differences.

SUMMARY

One AI model.
Three clouds.
One tool: cmx vm.

Questions?

github.com/cloudmesh-ai-luc/shabbir04

Project details: project.md